Lesson 05 Demo 06

Fetching Data Using a Custom Hook

Objective: To set up a React application using Vite and implement a custom Hook for fetching data efficiently, ensuring faster development and improved performance

Tools required: Visual Studio Code and Node.js

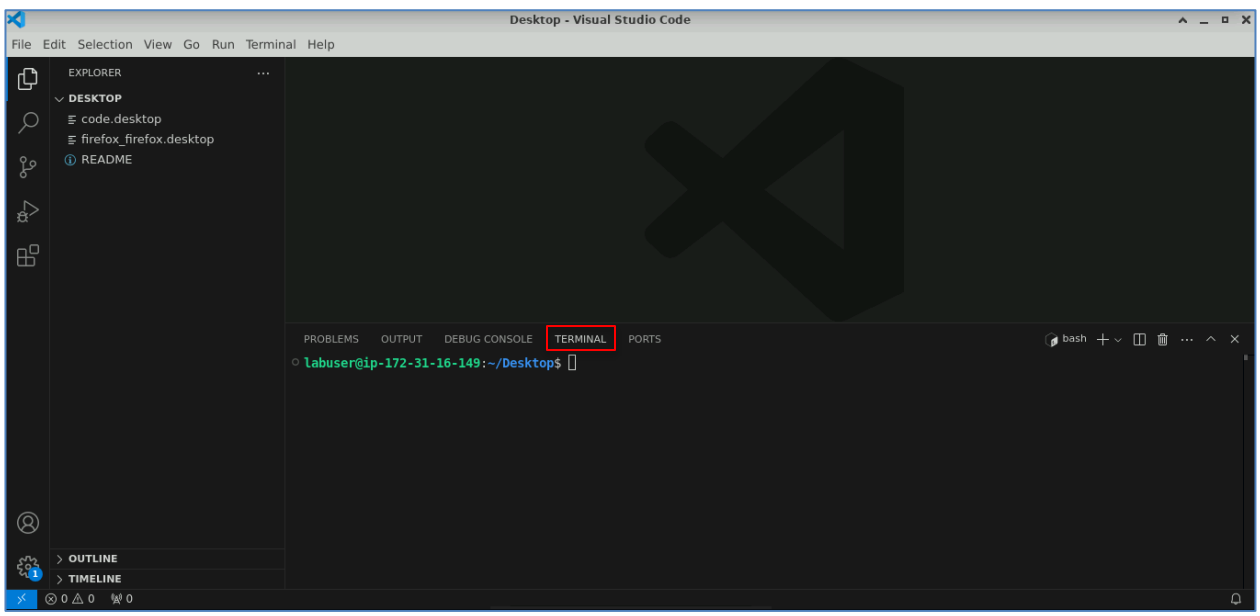
Prerequisites: None

Steps to be followed:

1. Set up a new React project using Vite
2. Create a custom Hook
3. Import and use the custom Hook
4. Run and verify the application

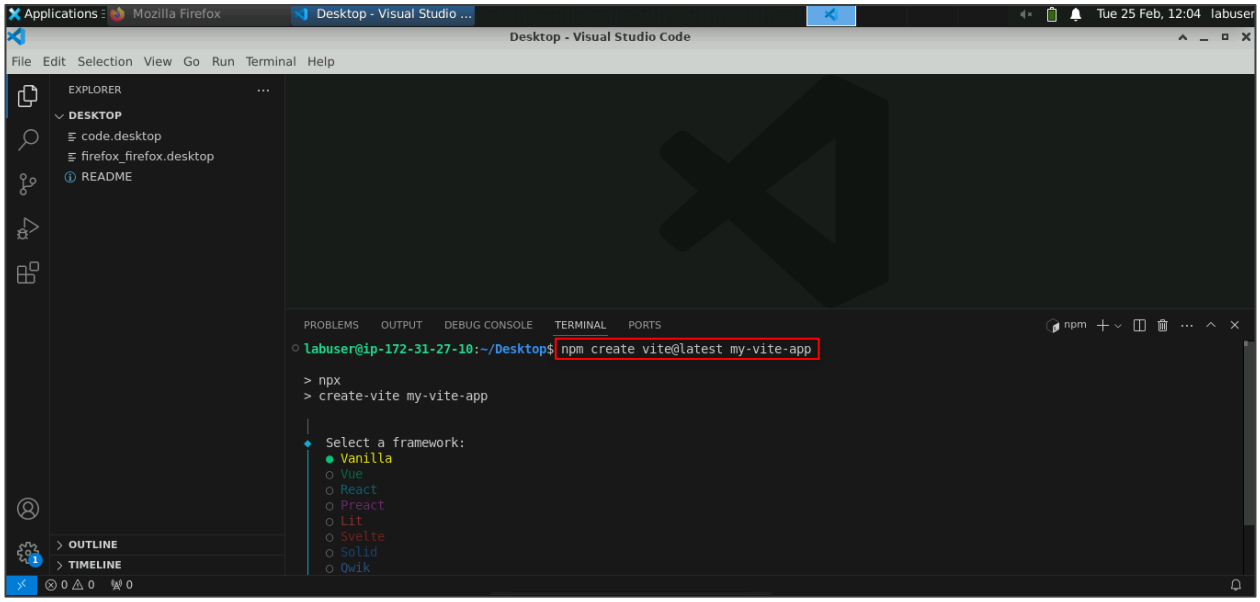
Step 1: Set up a new React project using Vite

1.1 Open Visual Studio Code (VS Code) and launch the terminal

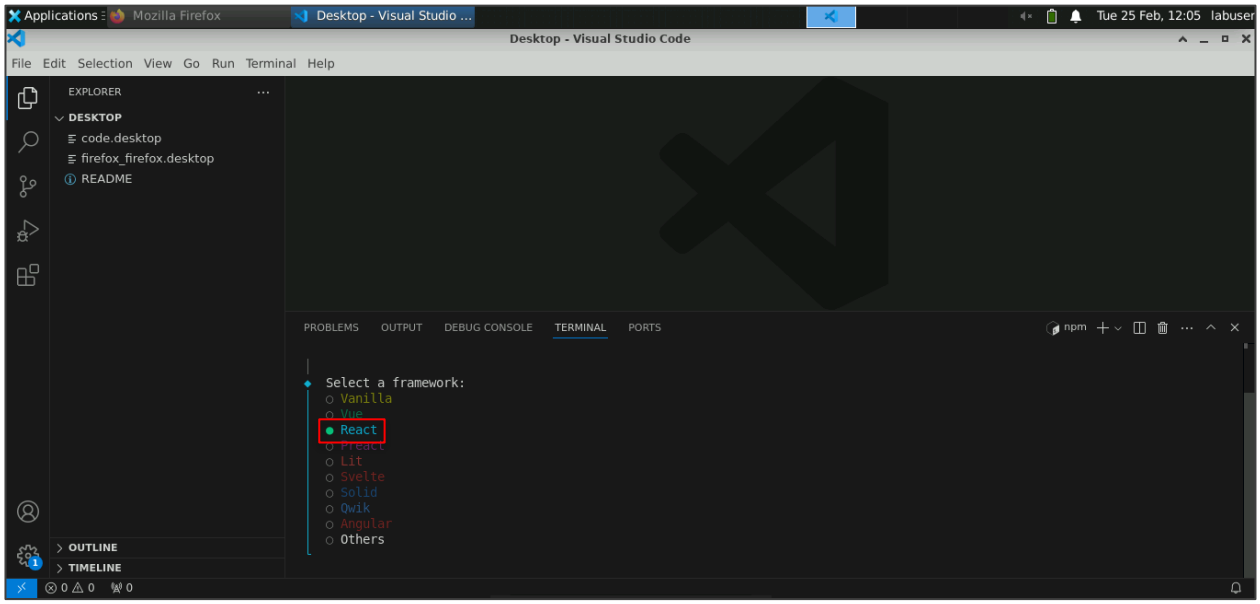


1.2 Run the following command to create a new React project using Vite:

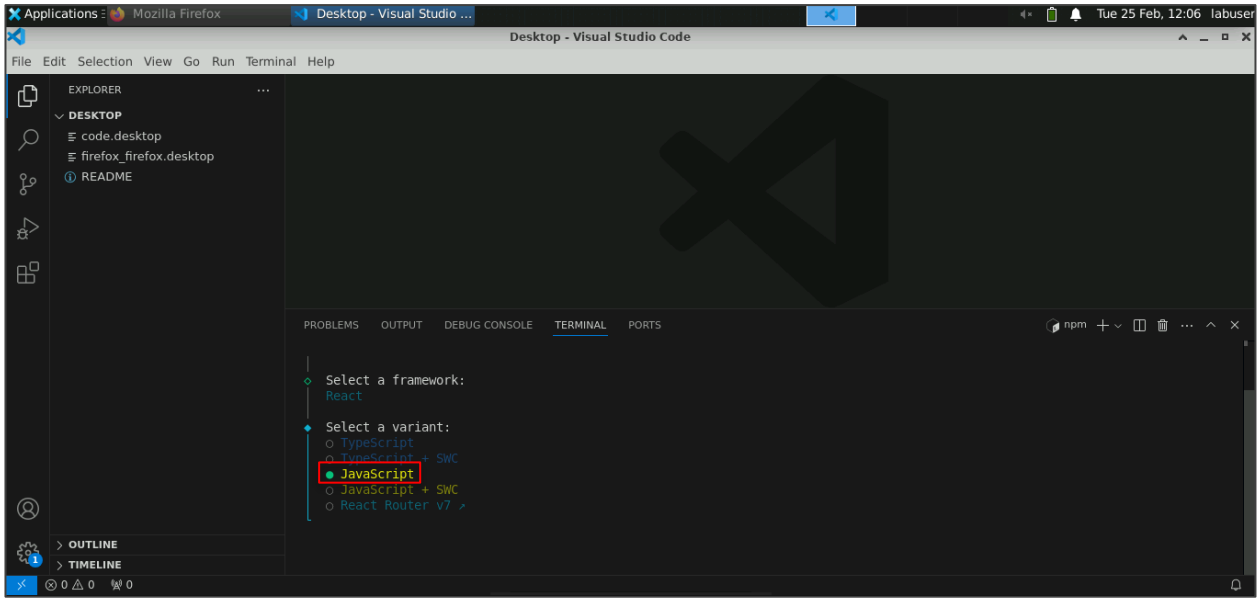
npm create vite@latest my-vite-app



1.3 Select **React** as the project framework and press enter

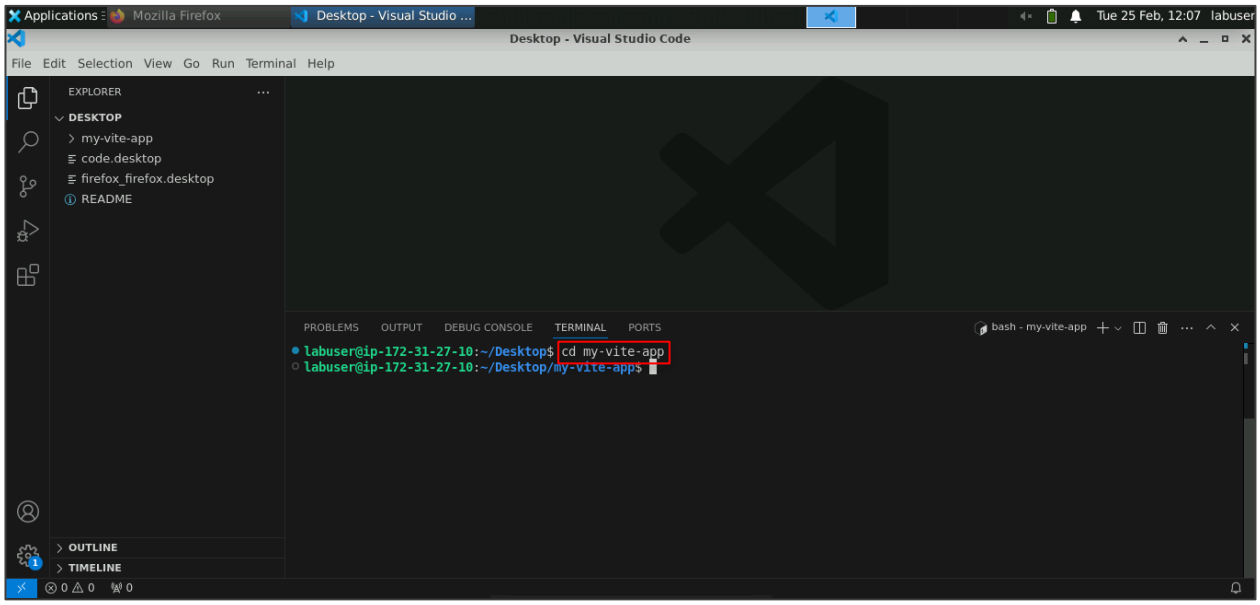


1.4 Select **JavaScript** as the language and press enter



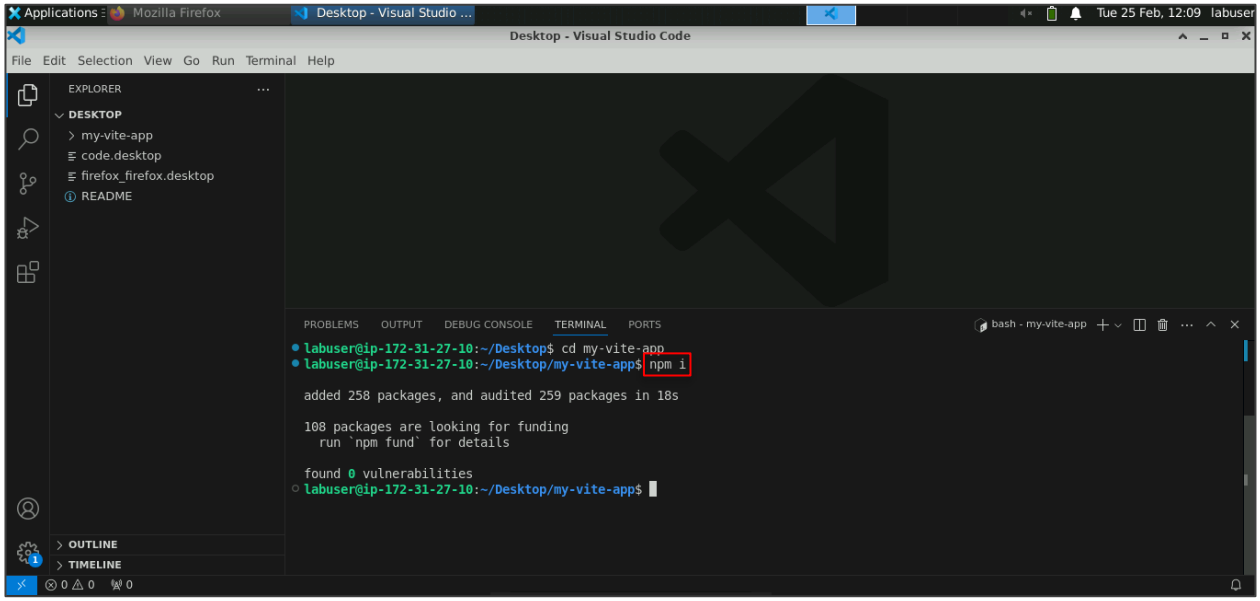
1.5 Run the following command to navigate into the project folder:

cd my-vite-app



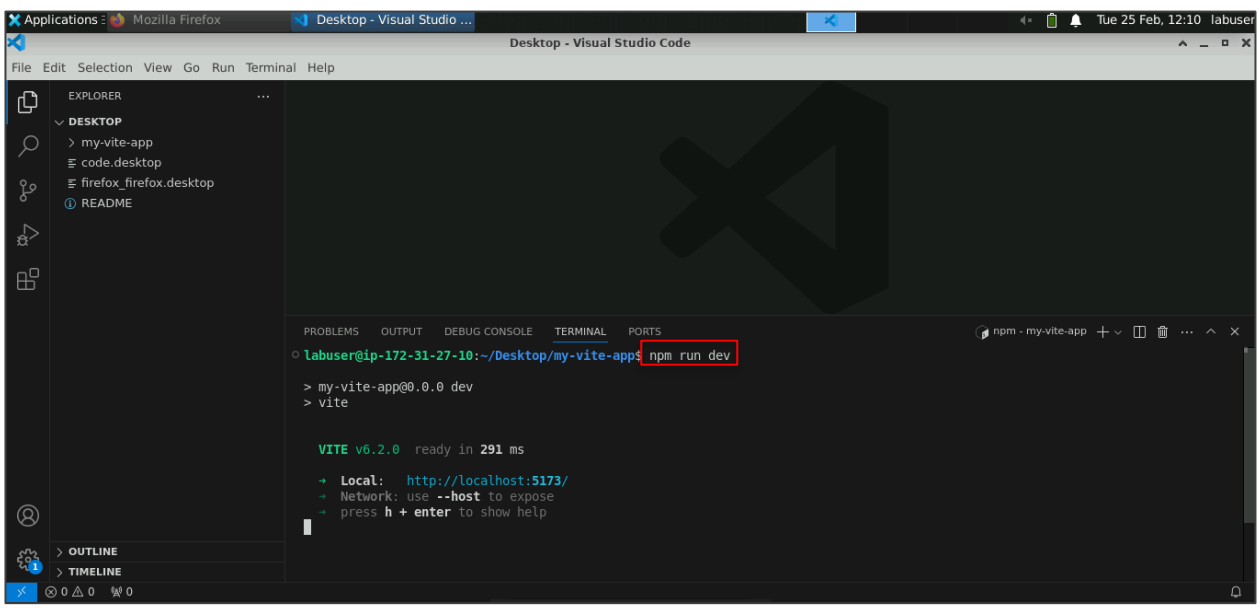
1.6 Run the following command to install dependencies:

npm i

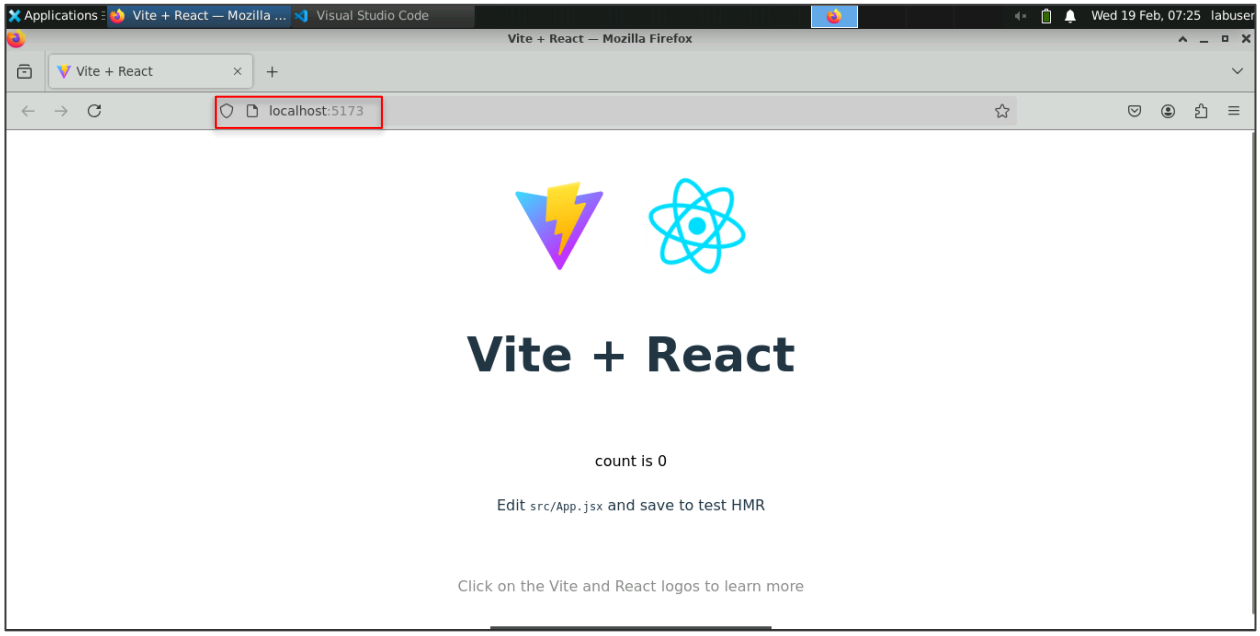


1.7 Run the following command to start the React development server:

npm run dev

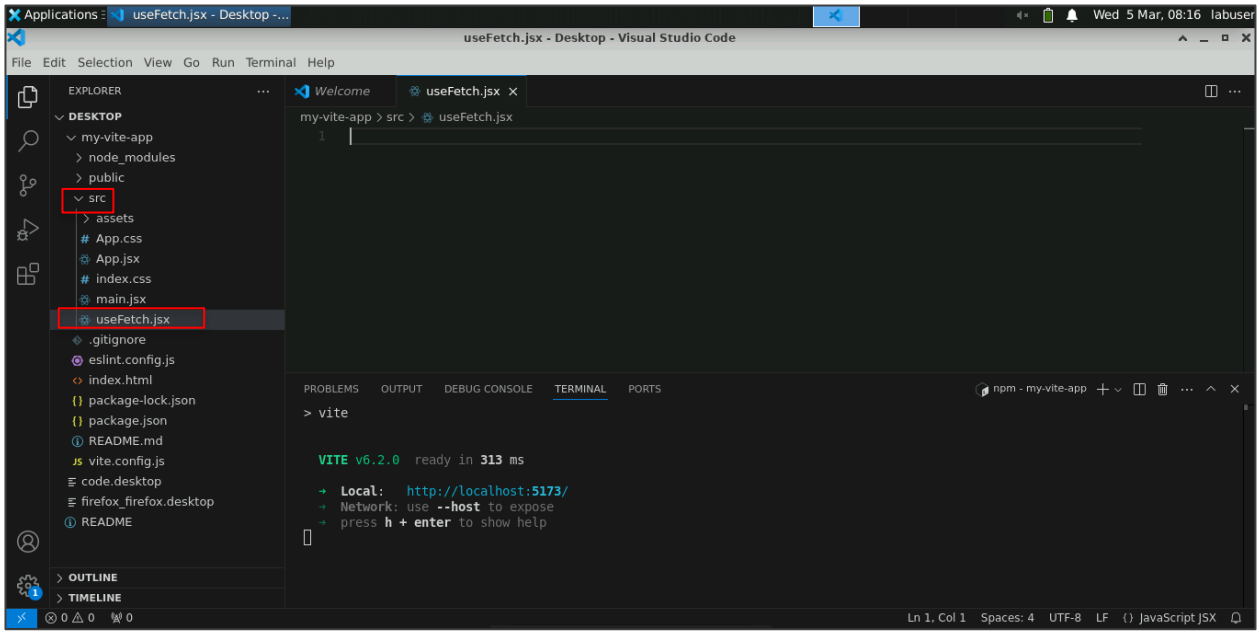


1.8 Navigate to the web browser and paste the following link:
http://localhost:5173



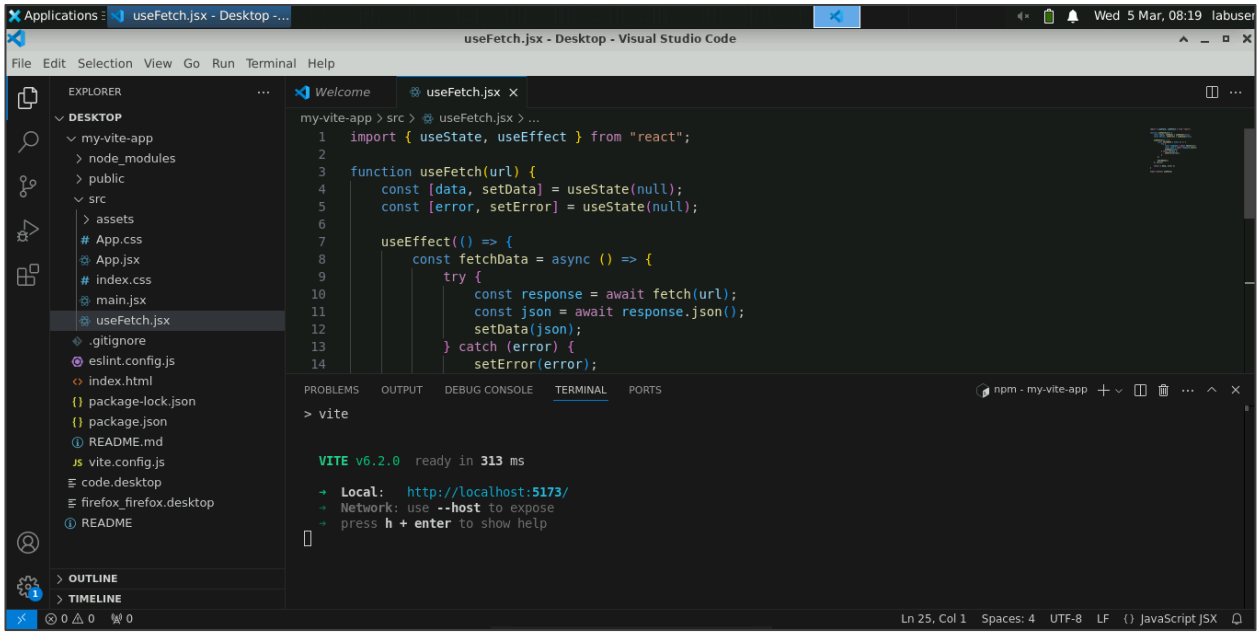
Step 2: Create a custom Hook

2.1 Inside the **src** folder, create a new file named **useFetch.jsx**



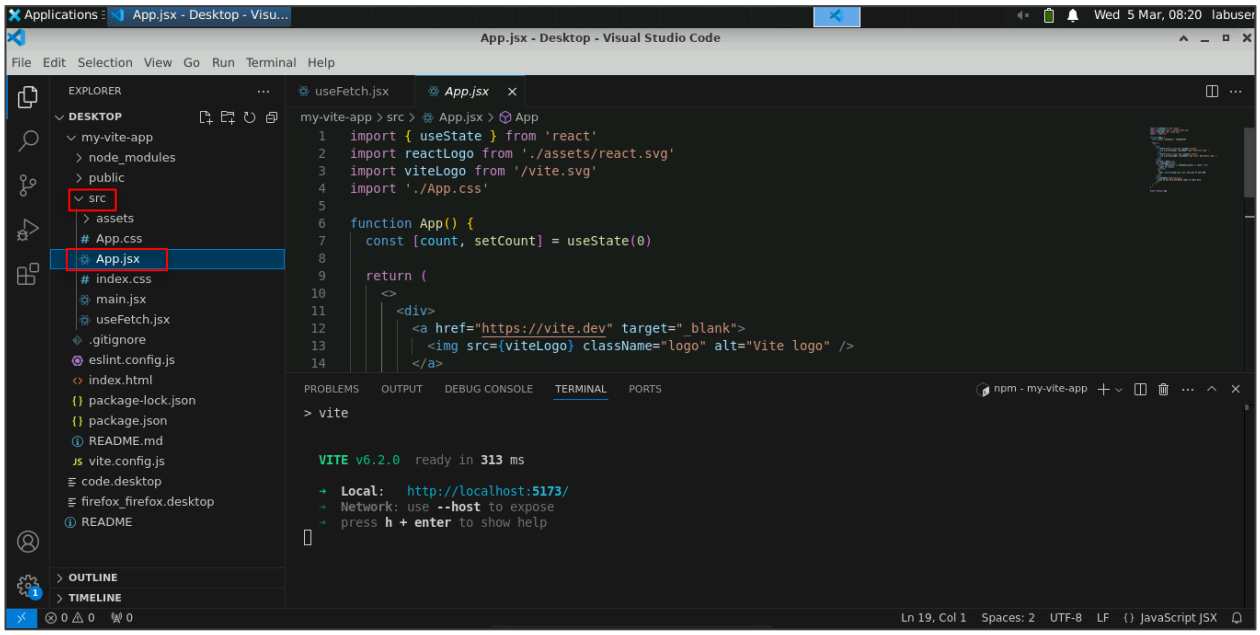
2.2 Add the following code to **useFetch.jsx** to create a custom Hook for fetching data:

```
import { useState, useEffect } from "react";

function useFetch(url) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch(url);
        const json = await response.json();
        setData(json);
      } catch (error) {
        setError(error);
      }
    };

    fetchData();
  }, [url]);

  return { data, error };
}

export default useFetch;
```



Step 3: Import and use the custom Hook

3.1 Open the **App.jsx** file inside the **src** folder



3.2 Paste the following code to include the new components in **App.jsx**:

```
import React from "react";
import useFetch from "./useFetch";
import "./App.css";

function App() {
  const { data, error } = useFetch("https://jsonplaceholder.typicode.com/todos/1");

  if (error) {
    return <div>Error: {error.message}</div>;
  }

  if (!data) {
    return <div>Loading...</div>;
  }

  return (
    <div className="App">
      <h1>Custom Hook Fetch Demo</h1>
      <p>Title: {data.title}</p>
      <p>Completed: {data.completed ? "Yes" : "No"}</p>
    </div>
  );
}

export default App;
```

Applications App.jsx - Desktop - Visu... Wed 5 Mar, 08:21 labuser

App.jsx - Desktop - Visual Studio Code

File Edit Selection View Go Run Terminal Help

EXPLORER

DESKTOP
- my-vite-app
  - node_modules
  - public
  - src
    - assets
    - App.css
    - App.jsx
    - index.css
    - main.jsx
    - useFetch.jsx
  - .gitignore
  - eslint.config.js
  - index.html
  - package-lock.json
  - package.json
  - README.md
  - vite.config.js
- code.desktop
- firefox_firefox.desktop
- README

OUTLINE

TIMELINE

useFetch.jsx | App.jsx

my-vite-app > src > App.jsx > ...

```jsx
import React from "react";
import useFetch from "./useFetch";
import "./App.css";

function App() {
    const { data, error } = useFetch("https://jsonplaceholder.typicode.com/todos/1");

    if (error) {
        return <div>Error: {error.message}</div>;
    }

    if (!data) {
        return <div>Loading...</div>;
    }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

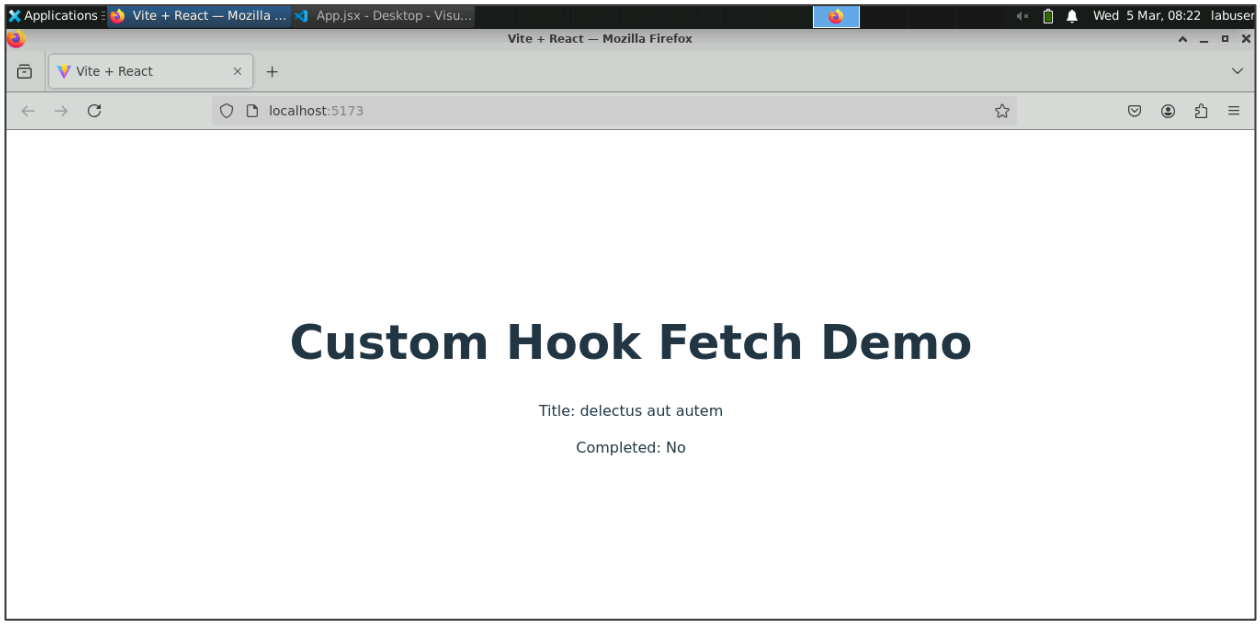
npm - my-vite-app

```
> vite


  VITE v6.2.0  ready in 313 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

0 0 0 Ln 26, Col 1 Spaces: 2 UTF-8 LF {} JavaScript JSX

Step 4: Run and verify the application

4.1 Open the web browser and refresh the page



The application should display the first entry from the **JSONPlaceholder API's** /todos/1 endpoint.

By following these steps, you have successfully set up a React application using Vite and implemented a custom Hook for efficient data fetching. This approach enhances modularity, reusability, and maintainability, ensuring seamless updates and an optimized development experience.